

I need a step by step guide on to achive this you'll be asking what step have I reached at to better solve this. I have already created the database . There was a setup wizad of some kind with options to even import the a ready .sql db. Should I open a new window to create the vpc? 'SmartShop Deployment: The Droplet (VPS)  
Strategy  
LUWI Mafuleti  
May 27, 2026  
Contents  
Show more ▾

You're already past one of the biggest steps: the database exists. Good.

About your VPC question first:

- **No, you do not need to open a new browser window to create the VPC.**
- But you **should create the VPC before creating the Droplet** if possible.
- If the Droplet is already created, you can still attach it to a VPC afterward from the networking settings.

For your setup, a VPC is mainly useful for:

- private networking,
- future scaling,
- isolating DB traffic,
- adding Redis or another server later.

Since your MySQL is on the same Droplet right now, the VPC is not critical, but it's still a good practice.

Here's the clean order you should follow.

## SmartShop Deployment — Guided Walkthrough

I'll guide you phase-by-phase, and after each section you tell me:

- what you already completed,
- what screen you're currently on,

- and any errors you see.

Then I'll give you the exact next commands.

## Phase 0 — What You Need Ready

Before continuing, confirm you have:

- GitHub repo for SmartShop
- Domain name ( `smartshop-luwi.tech` )
- SSH key added to DigitalOcean
- Existing SQL database export ( `.sql` ) if you already have data
- Droplet not yet fully configured

## Phase 1 — Create the VPC (Recommended)

In DigitalOcean:

### Step 1

Go to:

[DigitalOcean Control Panel](#)

### Step 2

Left sidebar:

Networking → VPC

### Step 3

Create VPC:

- Name: `smartshop-vpc`
- Region: SAME region as your future Droplet
  - Example: Frankfurt / London / Amsterdam

### Step 4

Save it.

That's it.

## Phase 2 — Create the Droplet

Now create the server.

### Recommended Settings

#### Image

- Ubuntu 24.04 LTS

#### Plan

- Basic
- Regular Intel
- \$6/month

#### Authentication

- SSH Key (NOT password)

#### Additional Options

Enable:

- Monitoring

Do NOT enable:

- backups (unless you want them)

#### Firewall

Allow:

- SSH (22)
- HTTP (80)
- HTTPS (443)

#### VPC

Select:

smartshop-vpc

## Phase 3 — Connect to the Server

Once the Droplet is created:

Copy the public IP.

On your computer terminal:

Bash

```
ssh root@YOUR_SERVER_IP
```

Example:

Bash

```
ssh root@159.89.xxx.xxx
```

If SSH key setup was correct:

- it logs in immediately.

## Phase 4 — Initial Server Setup

Run:

Bash

```
sudo apt update && sudo apt upgrade -y
```

Then:

Bash

```
sudo apt install -y git curl unzip zip nginx supervisor
```

## Phase 5 — Install PHP 8.3

Run these one-by-one:

Bash

```
sudo add-apt-repository ppa:ondrej/php -y
```

Then:

Bash

```
sudo apt update
```

Then:

Bash

```
sudo apt install -y php8.3-fpm php8.3-mysql php8.3-mbstring php8.3-xml php8.3-bcmath
```



Verify:

Bash

```
php -v
```

You should see PHP 8.3.x.

## Phase 6 — Install MySQL

Install:

Bash

```
sudo apt install -y mysql-server
```

Secure it:

Bash

```
sudo mysql_secure_installation
```

Recommended answers:

- validate password plugin → No
- remove anonymous users → Yes
- disallow remote root login → Yes
- remove test db → Yes

## IMPORTANT — Since You Already Created the Database

You said:

“There was a setup wizard with import SQL.”

That likely means:

- you already imported your `.sql`
- OR you already created tables.

So STOP before running:

Bash

```
php artisan migrate --force
```

because it may:

- duplicate tables,
- conflict,
- or damage existing schema.

Instead, first check whether your DB already has tables.

## Check Existing Database

Login:

Bash

```
sudo mysql
```

Then:

SQL

```
SHOW DATABASES;
```

Then:

SQL

```
USE smartshop;  
SHOW TABLES;
```

If tables already exist:

- DO NOT run migrations yet.

Exit MySQL:

```
SQL
```

```
exit
```

## If You Need to Import Existing SQL

Only if the DB is empty.

Example:

```
Bash
```

```
mysql -u root -p smartshop < backup.sql
```

## Phase 7 — Create Database User

If not already done:

```
Bash
```

```
sudo mysql
```

Then:

```
SQL
```

```
CREATE USER 'luwi'@'localhost' IDENTIFIED BY 'StrongPasswordHere';
```

```
GRANT ALL PRIVILEGES ON smartshop.* TO 'luwi'@'localhost';
```

```
FLUSH PRIVILEGES;
```

## Phase 8 — Clone Laravel Project

Move into web root:

```
Bash
```

```
cd /var/www
```

Clone repo:

Bash

```
sudo git clone https://github.com/YOUR_USERNAME/e-commerce-platform.git smartshop
```

Set permissions:

Bash

```
sudo chown -R www-data:www-data /var/www/smartshop
```

Then:

Bash

```
sudo chmod -R 775 /var/www/smartshop/storage  
sudo chmod -R 775 /var/www/smartshop/bootstrap/cache
```

## Phase 9 — Install Composer

Install:

Bash

```
curl -sS https://getcomposer.org/installer | php
```

Move binary:

Bash

```
sudo mv composer.phar /usr/local/bin/composer
```

Verify:

Bash

```
composer --version
```

## Phase 10 — Install Laravel Dependencies

Go into project:

Bash

```
cd /var/www/smartshop
```

Install dependencies:

Bash

```
composer install --no-dev --optimize-autoloader
```

## Phase 11 — Configure .env

Copy env:

Bash

```
cp .env.example .env
```

Edit:

Bash

```
nano .env
```

Update:

env

```
APP_ENV=production
APP_DEBUG=false
```

```
DB_DATABASE=smartshop
DB_USERNAME=luwi
DB_PASSWORD=YOUR_PASSWORD
```

Save:

- CTRL+O
- ENTER
- CTRL+X

Generate key:

Bash

```
php artisan key:generate
```

# STOP HERE BEFORE MIGRATIONS

Because your database may already contain tables.

First run:

Bash

```
php artisan migrate:status
```

This tells us:

- whether Laravel recognizes migrations,
- and whether tables already exist.

## Your Next Reply

Tell me:

1. Have you already:
  - created the Droplet?
  - created the VPC?
  - connected via SSH?
2. Did you already import the `.sql` database?
3. What happens when you run:

SQL

```
SHOW TABLES;
```

4. Are you currently:
  - on DigitalOcean dashboard,
  - terminal,
  - or inside MySQL?

Then I'll guide you through:

- Nginx configuration,
- domain setup,
- SSL,
- queue workers,
- Laravel optimization,
- and production hardening.



















































































This is a terminal/editor issue, not a Laravel problem.

nano is either:

- not installed properly, or
- your terminal type ( `xterm-kitty` ) is causing compatibility issues.

We'll fix it quickly.

## OPTION 1 (FASTEST FIX) — Install Nano

Run:

Bash

```
sudo apt update
sudo apt install -y nano
```

Then try again:

Bash

```
nano .env
```

## OPTION 2 — Use Vi (already installed on all Linux servers)

If you don't want nano, use:

Bash

```
vi .env
```

### Vi basics:

- press `i` → insert mode
- edit file
- press `ESC`
- type:

```
:wq
```

- press ENTER to save and exit

## OPTION 3 — Fix terminal issue (xterm-kitty)

If you keep seeing weird terminal behavior, force a safer terminal mode:

```
Bash
```

```
export TERM=xterm
```

Then retry:

```
Bash
```

```
nano .env
```

## RECOMMENDATION

Use OPTION 1 (nano install). It's simplest for deployment work.

## AFTER YOU EDIT .ENV

Run:

```
Bash
```

```
php artisan key:generate
```

```
php artisan --version
```

Then tell me:

- did `.env` open successfully?
- and output of `php artisan --version`

Next step will be:

👉 database import + Nginx setup + domain binding (this is where your app goes live).















Good — database side is now considered done. Next stage is what actually makes your project “go live”: **Nginx + domain + SSL + production hardening**.

We'll do this in a clean order so nothing breaks.

## PHASE 1 — Fix Laravel production permissions (IMPORTANT)

Run:

Bash

```
sudo chown -R www-data:www-data /var/www/smartshop
sudo chmod -R 775 /var/www/smartshop/storage
sudo chmod -R 775 /var/www/smartshop/bootstrap/cache
```

Then clear cache:

Bash

```
cd /var/www/smartshop
php artisan config:clear
php artisan cache:clear
```

## PHASE 2 — Install & configure Nginx site

### Step 1: Create config file

Bash

```
sudo nano /etc/nginx/sites-available/smartshop
```

Paste this:

Nginx

```
server {
    listen 80;
    server_name smartshop-luwi.tech www.smartshop-luwi.tech;

    root /var/www/smartshop/public;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

### Step 2: Enable site

Bash

```
sudo ln -s /etc/nginx/sites-available/smartshop /etc/nginx/sites-enabled/
```

Remove default site (important):

Bash

```
sudo rm /etc/nginx/sites-enabled/default
```

## Step 3: Test Nginx

Bash

```
sudo nginx -t
```

If OK:

Bash

```
sudo systemctl restart nginx
```

## PHASE 3 — DOMAIN SETUP (VERY IMPORTANT)

Go to your domain provider DNS and add:

### A Record:

Type: A

Name: @

Value: YOUR\_SERVER\_IP

Type: A

Name: www

Value: YOUR\_SERVER\_IP

Wait 5–30 minutes for propagation.

Test:

Bash

```
ping smartshop-luwi.tech
```

## PHASE 4 — SSL (HTTPS with Let's Encrypt)

Install Certbot:

Bash

```
sudo apt install -y certbot python3-certbot-nginx
```

Run SSL setup:

Bash

```
sudo certbot --nginx -d smartshop-luwi.tech -d www.smartshop-luwi.tech
```

Choose:

- redirect HTTP → HTTPS → YES

## PHASE 5 — FINAL LARAVEL PRODUCTION OPTIMIZATION

Bash

```
php artisan config:cache  
php artisan route:cache  
php artisan view:cache
```

## PHASE 6 — VERIFY EVERYTHING

Open in browser:

```
http://smartshop-luwi.tech
```

Then:

```
https://smartshop-luwi.tech
```

You should see your app live.

## IF SOMETHING BREAKS (COMMON ISSUES)

### 1. 502 Bad Gateway

→ PHP-FPM not running

Bash

```
sudo systemctl restart php8.3-fpm
```

## 2. Blank page

→ Laravel error logs:

Bash

```
tail -f storage/logs/laravel.log
```

## 3. Permission errors

→ re-run:

Bash

```
sudo chown -R www-data:www-data /var/www/smartshop
```

## NEXT STEP AFTER THIS (FINAL LEVEL)

After this we can upgrade your deployment with:

- Supervisor (queues)
- Cron scheduler
- Redis cache (optional)
- GitHub auto-deploy (CI/CD)
- Performance tuning

## YOUR ACTION NOW

Do this first:

👉 Nginx config + enable site + test

Then tell me:

- “nginx done”
- or paste any error output

and I'll take you to the SSL + final live deployment stage.



It worked first before updating the script, I want to say something. The emails need a php artisan work something like this needs to be always running what do i do?

Yes — you're thinking about the right problem. Email sending (and most background work in Laravel) should **not run inside the web request**. It should be handled by a **queue worker** that runs continuously.

In Laravel, that's done with:

- `php artisan queue:work` (worker)
- or a daemon manager like **Supervisor** (production way)

Right now you're in the classic stage where:

“It works manually, but production needs it always running”

Let's fix it properly.

## OPTION YOU SHOULD USE (PRODUCTION STANDARD): SUPERVISOR

This keeps your queue worker alive even after:

- SSH disconnects
- server reboot
- crashes

## STEP 1 — Check your queue driver

Open `.env` :

Bash

```
nano /var/www/smartshop/.env
```

Look for:

env

```
QUEUE_CONNECTION=database
```

If it says `sync` , change it to:

env

```
QUEUE_CONNECTION=database
```

## STEP 2 — Make sure queue tables exist

If you haven't already:

Bash

```
php artisan queue:table  
php artisan migrate
```

(This creates the jobs table)

## STEP 3 — Install Supervisor

Bash

```
sudo apt update  
sudo apt install -y supervisor
```

## STEP 4 — Create Supervisor config

Bash

```
sudo nano /etc/supervisor/conf.d/smartshop-worker.conf
```

Paste:

INI

```
[program:smartshop-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/smartshop/artisan queue:work --sleep=3 --tries=3 --timeout=90
autostart=true
autorestart=true
user=www-data
numprocs=1
redirect_stderr=true
stdout_logfile=/var/www/smartshop/storage/logs/worker.log
stopwaitsecs=3600
```

## STEP 5 — Activate it

Bash

```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start smartshop-worker:*
```

## STEP 6 — Check if it's running

Bash

```
sudo supervisorctl status
```

You should see:

```
smartshop-worker:smartshop-worker_00    RUNNING
```

## WHAT THIS SOLVES (IMPORTANT)

Now your system can:

- Send emails in background
- Process orders without delay
- Handle PayPal/webhooks cleanly
- Run notifications, invoices, etc.

## OPTIONAL (BETTER SETUP LATER)

If you want “enterprise level”:

- Redis queue (faster)
- Horizon dashboard (monitoring)
- Separate workers for emails vs orders

But not needed yet — Supervisor is perfect for your stage.

## QUICK REALITY CHECK

Right now your architecture becomes:

- Nginx → handles requests
- PHP-FPM → executes Laravel
- Supervisor → runs queue worker forever

That’s a proper production setup.

## NEXT QUESTION FOR YOU

Before we move to the next improvement:

Do you want emails to be:

1. ⚡ Instant (send immediately, simple but slower requests)
2. 🚀 Queue-based (recommended, what we just set up)
3. 🔥 Hybrid (emails queued, critical ones instant)

Tell me, and I’ll align your system properly + help you test an email flow end-to-end.



It worked, Now I need a latex section of everything we have done especially the nano script should be as they are this is for documentation purposes. Can you help em? I need a concrete document. or rather update this and give me ready script

```
\documentclass[12pt,a4paper]{article}
\usepackage[utf8]{inputenc}
\usepackage[T1]{fontenc}
```

```
\usepackage{geometry}
\geometry{margin=1in}
\usepackage{hyperref}

Show more ▾
```

Yes — this is exactly the kind of document you want for a real deployment record. I've updated your LaTeX to include everything you actually did in the correct order, including:

- SQL import
- Laravel fixes
- `.env` editing via nano (with terminal fix)
- MySQL issue + recovery context
- Composer safe directory fix
- Queue worker (Supervisor) setup
- Real production state (not theoretical steps)

Here is your **clean, production-grade final LaTeX document**:

LaTeX

```
\documentclass[12pt,a4paper]{article}
\usepackage[utf8]{inputenc}
\usepackage[T1]{fontenc}
\usepackage{geometry}
\geometry{margin=1in}
\usepackage{hyperref}
\usepackage{xcolor}
\usepackage{listings}

\definecolor{brandblue}{RGB}{30, 64, 175}
\definecolor{codebg}{RGB}{245, 245, 245}
\hypersetup{colorlinks=true, linkcolor=brandblue, urlcolor=brandblue}

\lstset{
  backgroundcolor=\color{codebg},
  basicstyle=\ttfamily\small,
  breaklines=true,
  frame=single,
  keywordstyle=\color{blue},
  commentstyle=\color{gray}
}

\title{\textbf{SmartShop Deployment: Production VPS Implementation Report}}
\author{LUWI Mafuleti}
```

```
\date{\today}

\begin{document}

\maketitle
\tableofcontents
\newpage

\section{Introduction}
This document outlines the full deployment process of the SmartShop Laravel applicati

\section{Phase 1: Droplet Provisioning}
\begin{itemize}
    \item Ubuntu 24.04 LTS deployed on DigitalOcean Droplet
    \item SSH key authentication configured
    \item Firewall enabled (SSH 22, HTTP 80, HTTPS 443)
\end{itemize}

\section{Phase 2: Server Setup (LEMP Stack)}

\subsection{System Update}
\begin{lstlisting}[language=bash]
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl unzip zip nginx supervisor
\end{lstlisting}

\subsection{PHP 8.3 Installation}
\begin{lstlisting}[language=bash]
sudo add-apt-repository ppa:ondrej/php -y
sudo apt update
sudo apt install -y php8.3-fpm php8.3-mysql php8.3-mbstring php8.3-xml php8.3-bcmath
\end{lstlisting}

\subsection{MySQL Installation and Recovery}
During installation, MySQL failed initially due to memory constraints (OOM kill). The

\begin{lstlisting}[language=bash]
sudo apt install -y mysql-server
sudo systemctl enable mysql
sudo systemctl start mysql
\end{lstlisting}

\subsection{Database Setup}
\begin{lstlisting}[language=bash]
sudo mysql -u root
CREATE DATABASE smartshop;
CREATE USER 'luwi'@'localhost' IDENTIFIED BY 'secure_password';
GRANT ALL PRIVILEGES ON smartshop.* TO 'luwi'@'localhost';
FLUSH PRIVILEGES;
\end{lstlisting}

\subsection{SQL Import}
```

The existing project database was imported using:

```
\begin{lstlisting}[language=bash]
mysql -u luwi -p smartshop < database_dump.sql
\end{lstlisting}
```

\section{Phase 3: Application Deployment}

\subsection{Repository Clone}

```
\begin{lstlisting}[language=bash]
cd /var/www
git clone https://github.com/yourusername/e-commerce-platform.git smartshop
cd smartshop
\end{lstlisting}
```

\subsection{Fixing Git Safe Directory Issue}

```
\begin{lstlisting}[language=bash]
git config --global --add safe.directory /var/www/smartshop
\end{lstlisting}
```

\subsection{Permissions}

```
\begin{lstlisting}[language=bash]
sudo chown -R www-data:www-data /var/www/smartshop
sudo chmod -R 775 /var/www/smartshop/storage
sudo chmod -R 775 /var/www/smartshop/bootstrap/cache
\end{lstlisting}
```

\subsection{Composer Installation}

```
\begin{lstlisting}[language=bash]
composer install --no-dev --optimize-autoloader
\end{lstlisting}
```

\textbf{Note:} Composer required safe directory configuration due to root ownership.

\section{Phase 4: Environment Configuration}

\subsection{Nano Terminal Fix}

The terminal initially failed with:

```
\texttt{Error opening terminal: xterm-kitty}
```

Resolved using:

```
\begin{lstlisting}[language=bash]
export TERM=xterm
\end{lstlisting}
```

\subsection{Laravel Environment Setup}

```
\begin{lstlisting}[language=bash]
cp .env.example .env
nano .env
php artisan key:generate
\end{lstlisting}
```

Database configuration:

```
\begin{lstlisting}
DB_DATABASE=smartshop
DB_USERNAME=luwi
DB_PASSWORD=secure_password
\end{lstlisting}
```

\section{Phase 5: Laravel Optimization}

```
\begin{lstlisting}[language=bash]
php artisan config:clear
php artisan cache:clear
php artisan config:cache
\end{lstlisting}
```

\section{Phase 6: Nginx Configuration}

```
\begin{lstlisting}[language=bash]
sudo nano /etc/nginx/sites-available/smartshop
sudo ln -s /etc/nginx/sites-available/smartshop /etc/nginx/sites-enabled/
sudo systemctl restart nginx
\end{lstlisting}
```

Nginx root set to:

```
\texttt{/var/www/smartshop/public}
```

\section{Phase 7: Domain and SSL}

```
\begin{itemize}
\item Domain configured: smartshop-luwi.tech
\item DNS A records pointed to Droplet IP
\item SSL issued using Certbot
\end{itemize}
```

```
\begin{lstlisting}[language=bash]
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d smartshop-luwi.tech
\end{lstlisting}
```

\section{Phase 8: Background Job Processing (Queues)}

\subsection{Queue Configuration}

```
\begin{lstlisting}
QUEUE_CONNECTION=database
\end{lstlisting}
```

\subsection{Supervisor Setup}

```
\begin{lstlisting}[language=bash]
sudo apt install -y supervisor
sudo nano /etc/supervisor/conf.d/smartshop-worker.conf
\end{lstlisting}
```

Configuration:

```
\begin{lstlisting}
[program:smartshop-worker]
command=php /var/www/smartshop/artisan queue:work --sleep=3 --tries=3 --timeout=90
autostart=true
autorestart=true
user=www-data
redirect_stderr=true
stdout_logfile=/var/www/smartshop/storage/logs/worker.log
\end{lstlisting}
```

Activation:

```
\begin{lstlisting}[language=bash]
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start smartshop-worker:*
\end{lstlisting}
```

\section{Conclusion}

The SmartShop application is now fully deployed on a production-grade VPS environment

```
\begin{itemize}
  \item LEMP stack configured
  \item Database imported and active
  \item Laravel optimized for production
  \item Nginx + SSL secured domain
  \item Background queue worker running via Supervisor
\end{itemize}
```

This completes the full transition from development to production infrastructure.

\end{document}

If you want next upgrade, I can help you add:

- architecture diagram (PDF-ready)
- CI/CD GitHub auto deploy section
- monitoring section (logs, uptime, alerts)
- performance benchmarking section

Just tell me 👍

